# ADVANCED WEB TECHNOLOGIES

Practical Exam Notes + Viva Question Bank

**Last-minute revision guide | Assignments 01–04**
JavaScript ES6+, arrays, OOP, DOM, localStorage, Fetch API, Promises, closures, React, JSX, Hooks, forms, Router, Redux, Error Boundaries, JWT, terminal commands and viva answers.

**Exam strategy:** Identify the input, operation and output. Write the smallest working example, test a normal case and an edge case, then explain the key concept.

These are compact revision templates. Adapt fields, labels and UI to the exact question.

# 1. Practical workflow and terminal commands

## Six-step approach

• Read the task and identify the key concept: map, class, DOM, useState, route, reducer, etc.

• Identify the data: primitive, array, object, component state or Redux state.

• Build the minimum required input → operation → output.

• Test an edge case: empty input, invalid number, missing item or negative count.

• Keep the browser console/terminal visible and fix the first error first.

• Explain what each important function, event handler or Hook does.

## Commands to know

```
node -v                         # Check Node.js version
npm -v                          # Check npm version
pwd                             # Current directory (Git Bash/macOS/Linux)
dir                             # List files (Windows CMD/PowerShell)
ls                              # List files (Git Bash/macOS/Linux)
cd folder                       # Move into a folder
cd ..                           # Move to parent folder
mkdir my-folder                 # Create a folder
clear                           # Clear Git Bash/macOS/Linux terminal
cls                             # Clear Windows CMD/PowerShell
code .                          # Open current folder in VS Code
npm create vite@latest my-app -- --template react
cd my-app
npm install                     # Install dependencies
npm run dev                     # Start Vite dev server
npm install react-router-dom    # Add routing
npm install @reduxjs/toolkit react-redux
npm run build                   # Production build
# Ctrl+C stops a running dev server
```

**Remember:** Run npm commands in the directory containing package.json. Basic HTML/CSS/JS can run directly in a browser. DOM and localStorage code must be tested in a browser, not Node alone.

## Typical React structure

```
my-app/
  index.html
  package.json
  src/
    main.jsx       # Entry point
    App.jsx        # Root component
    App.css
    components/    # Reusable components
```

# 2. JavaScript fundamentals and array methods

## Variables and types

```
let age = 21;            // block-scoped; can reassign
const name = "Asha";     // block-scoped; no reassignment
var oldWay = true;       // function-scoped; avoid in modern JS

// Types: string, number, boolean, undefined, null, object
const marks = 76;
const passed = marks >= 40;
```

• Use const by default; use let when reassignment is needed. const prevents rebinding, not mutation of object contents.

• Prefer === (strict equality) to ==.

• Falsy values include false, 0, empty string, null, undefined and NaN.

## Functions, arrays and objects

```
function add(a, b) { return a + b; }
const multiply = (a, b) => a * b;

const students = [
  { id: 1, name: "Asha", marks: 78 },
  { id: 2, name: "Ravi", marks: 35 },
  { id: 3, name: "Neha", marks: 91 }
];
students.push({ id: 4, name: "Om", marks: 66 });
```

## Array methods — know the difference

| Method | Purpose | Result |
|---|---|---|
| forEach() | Run callback for each item | No transformed array |
| map() | Transform each item | New array |
| filter() | Keep matching items | New filtered array |
| reduce() | Accumulate into one result | Number/object/etc. |
| find() | Return first match | Item or undefined |
| sort() | Sort in place | Reordered original array |

```
const passed = students.filter(s => s.marks >= 40);
const names = students.map(s => s.name);
const total = students.reduce((sum, s) => sum + s.marks, 0);
const topper = students.find(s => s.marks === 91);
const descending = [...students].sort((a,b) => b.marks - a.marks);

// ES6 syntax
const { name, marks } = students[0];        // destructuring
const changed = { ...students[0], marks: 85 }; // spread
const greeting = (user = "Guest") => `Hello ${user}`;
```

**Sort:** (a,b) => a-b sorts numbers ascending; b-a sorts descending. sort() mutates the array; [...arr].sort(...) sorts a shallow copy.

# 3. JavaScript OOP, inheritance and closures

## Class, constructor, getter, setter and inheritance

```
class Employee {
  constructor(name, salary) {
    this.name = name;
    this._salary = salary;
  }
  get salary() { return this._salary; }
  set salary(value) {
    if (value >= 0) this._salary = value;
  }
  details() { return `${this.name}: ${this.salary}`; }
}

class Manager extends Employee {
  constructor(name, salary, teamSize) {
    super(name, salary); // parent constructor
    this.teamSize = teamSize;
  }
}
const emp = new Employee("Asha", 45000);
emp.salary = 50000;
```

• Class: blueprint for creating objects. Constructor: initializes a new object.

• this refers to the current object, depending on how the function is called.

• extends creates a subclass; super() invokes the parent constructor.

• Getter/setter provide property-like access with controlled reading/assignment.

• Encapsulation bundles data and methods; closures/private fields can restrict direct access.

## Closure: private cart state

```
function createCart() {
  let items = []; // private variable
  return {
    add(name, price) { items.push({ name, price }); },
    remove(name) { items = items.filter(x => x.name !== name); },
    total() { return items.reduce((sum, x) => sum + x.price, 0); }
  };
}
const cart = createCart();
cart.add("Book", 300);
console.log(cart.total()); // 300
```

**Closure:** a function retains access to variables in its lexical outer scope even after the outer function has finished. This is useful for private state.

## Callback and higher-order function

```
function processUser(name, callback) {
  callback(name);
}
processUser("Asha", user => console.log(`Welcome ${user}`));
```

A higher-order function accepts a function as an argument or returns a function. A callback is the function passed in.

# 4. DOM manipulation and localStorage

## DOM: select, listen, create, append

```
// HTML: <input id="taskInput"><button id="addBtn">Add</button><ul id="list"></ul>
const input = document.getElementById("taskInput");
const list = document.getElementById("list");

document.getElementById("addBtn").addEventListener("click", () => {
  const value = input.value.trim();
  if (!value) return;
  const li = document.createElement("li");
  li.textContent = value;
  list.appendChild(li);
  input.value = "";
});
```

| API | Purpose |
|---|---|
| getElementById() | Select by id. |
| querySelector() | Select first matching CSS selector. |
| querySelectorAll() | Select all matching elements. |
| addEventListener() | Attach an event handler. |
| createElement() | Create an element. |
| appendChild() | Append a child node. |
| remove() | Remove an element. |
| classList.add/remove/toggle() | Modify CSS classes. |

## Form submit

```
form.addEventListener("submit", event => {
  event.preventDefault(); // prevent page reload
  const email = document.querySelector("#email").value.trim();
  console.log(email);
});
```

## localStorage and JSON

```
const tasks = ["Revise JS", "Revise React"];
localStorage.setItem("tasks", JSON.stringify(tasks));

const saved = JSON.parse(localStorage.getItem("tasks") || "[]");
localStorage.setItem("tasks", JSON.stringify([...saved, "Practice viva"]));
localStorage.removeItem("tasks"); // remove one key
localStorage.clear();             // clear all keys for this origin
```

• localStorage stores string key-value pairs and persists across browser sessions.

• Use JSON.stringify() to convert objects/arrays to strings; JSON.parse() converts JSON text back.

• sessionStorage is session/tab scoped; localStorage persists longer.

• Do not store passwords or sensitive secrets in localStorage.

**Task list pattern:** read input → validate → create element → append → update stored array. Expense tracker: store transaction objects → filter → reduce totals → render table. Movie booking: update selected seats → calculate seats × price → update summary.

# 5. Fetch API, Promises and async/await

| Term | Meaning |
|---|---|
| Synchronous | Statements execute in sequence; the current operation blocks the next in that flow. |
| Asynchronous | A task can complete later without blocking the rest of the program. |
| Promise | Represents a future result: pending, fulfilled or rejected. |
| then() / catch() | Handle fulfillment / rejection. |
| async | Declares a function that returns a Promise. |
| await | Waits for a Promise inside an async function. |

## Promise example

```
const transaction = new Promise((resolve, reject) => {
  const success = true;
  if (success) resolve("Transaction successful");
  else reject(new Error("Transaction failed"));
});
transaction.then(message => console.log(message))
           .catch(error => console.error(error.message));
```

## Fetch with error handling

```
async function getWeather(city) {
  try {
    const response = await fetch(
      `https://api.example.com/weather?city=${encodeURIComponent(city)}`
    );
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("Unable to load weather:", error.message);
  }
}
```

**Important:** fetch() generally rejects on network errors, not just because the server returns 404 or 500. Check response.ok and throw for unsuccessful HTTP responses.

• Callback: function passed to another function; deeply nested callbacks can be hard to manage.

• Promise: structured handling of future success/failure.

• async/await: readable syntax built on Promises.

• Banking demo: deposits/withdrawals must be positive; withdrawals/transfers must not exceed balance; handle failures with try/catch. A browser-only banking task is a simulation, not a secure banking system.

# 6. React fundamentals: JSX, components and props

## Function component and JSX

```
function Welcome() {
  return (
    <div>
      <h1>Welcome to AWT</h1>
      <p>My first React component.</p>
    </div>
  );
}
export default Welcome;
```

• React is a JavaScript library for building component-based user interfaces.

• A component is a reusable piece of UI; function components are standard.

• JSX is HTML-like syntax in JavaScript, transformed into JavaScript.

• Component names start with a capital letter. Use className instead of class.

• A component returns one parent element or a Fragment <>....

• JavaScript expressions in JSX go inside curly braces: {expression}.

## Props and component composition

```
function BookCard({ title, author }) {
  return <article><h3>{title}</h3><p>{author}</p></article>;
}
function BookStore() {
  return (
    <main>
      <h1>Featured books</h1>
      <BookCard title="Clean Code" author="Robert Martin" />
    </main>
  );
}
```

Props pass data from parent to child and are read-only. Composition builds a larger UI from smaller components such as Header, BookCard, EmployeeCard and Footer.

## Lists and conditional rendering

```
const names = ["Asha", "Ravi", "Neha"];
function StudentList() {
  return <ul>{names.map(name => <li key={name}>{name}</li>)}</ul>;
}
const greeting = new Date().getHours() < 12 ? "Good Morning" : "Good Evening";
```

**Keys:** stable unique keys help React identify list items across updates. Avoid array indexes as keys when items can be inserted, deleted or reordered.

# 7. React state, Hooks, lifecycle and forms

## useState counter

```
import { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </>
  );
}
```

useState returns the current value and a setter. Calling the setter schedules a re-render. Do not mutate state arrays/objects directly; create new ones.

## Visitor counter and useEffect

```
const [entered, setEntered] = useState(0);
const [exited, setExited] = useState(0);
const current = entered - exited;

function leave() {
  if (current > 0) setExited(prev => prev + 1);
}
useEffect(() => {
  console.log("Visitor count changed:", current);
}, [current]);
```

| Effect dependency | When it runs |
|---|---|
| No second argument | After every render. |
| [] | After initial mount; cleanup on unmount. |
| [count] | After mount and when count changes. |

For timers/listeners, return a cleanup function from useEffect to clear them when the component unmounts or before the effect reruns.

## Class lifecycle — componentDidMount

```
class Clock extends React.Component {
  state = { time: new Date().toLocaleTimeString() };
  componentDidMount() {
    this.timer = setInterval(() => {
      this.setState({ time: new Date().toLocaleTimeString() });
    }, 1000);
  }
  componentWillUnmount() { clearInterval(this.timer); }
  render() { return <h2>{this.state.time}</h2>; }
}
```

componentDidMount() runs after mounting. componentWillUnmount() runs before removal. Clear intervals to avoid leaks.

## Controlled form validation

```
const [name, setName] = useState("");
const [email, setEmail] = useState("");
const [mobile, setMobile] = useState("");
const [error, setError] = useState("");

function submit(e) {
  e.preventDefault();
  if (!name.trim()) return setError("Name is required");
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email))
    return setError("Enter a valid email");
  if (!/^\d{10}$/.test(mobile))
    return setError("Mobile must contain 10 digits");
  setError("");
  alert("Submitted");
}
// JSX: <input value={name} onChange={e => setName(e.target.value)} />
```

• Controlled input: React state is the source of truth via value/checked and onChange.

• Use event.preventDefault() to stop normal page reload on form submit.

• Radio: checked/onChange; dropdown: value/onChange; textarea: value/onChange.

• Assignment checks: non-empty name, valid email, 10-digit mobile, percentage between 0 and 100; show errors and block invalid submission.

# 8. React Router and Redux

## React Router — dynamic route

```
import { BrowserRouter, Routes, Route, Link, useParams } from "react-router-dom";

function StudentDetails() {
  const { id } = useParams();
  return <h2>Student ID: {id}</h2>;
}
function App() {
  return (
    <BrowserRouter>
      <nav><Link to="/">Home</Link><Link to="/students">Students</Link></nav>
      <Routes>
        <Route path="/" element={<h1>Home</h1>} />
        <Route path="/students" element={<h1>Students</h1>} />
        <Route path="/students/:id" element={<StudentDetails />} />
        <Route path="/about" element={<h1>About</h1>} />
        <Route path="*" element={<h1>404 - Not found</h1>} />
      </Routes>
    </BrowserRouter>
  );
}
```

• BrowserRouter enables client-side routing; Routes contains Route definitions.

• Link navigates without a full page reload.

• :id is a dynamic route segment; useParams() reads it.

• Wildcard * route is commonly used for a 404 page.

## Redux vocabulary

| Term | Meaning |
|---|---|
| Store | Central container for application state. |
| Action | Object describing what happened; often type and payload. |
| Reducer | Calculates next state from current state and action. |
| Dispatch | Sends an action to the store. |
| Selector | Reads a value from the store. |
| Slice | Redux Toolkit grouping of reducer logic and generated actions. |

## Redux Toolkit expense slice

```
import { createSlice, configureStore } from "@reduxjs/toolkit";

const expenseSlice = createSlice({
  name: "expenses",
  initialState: [],
  reducers: {
    addExpense: (state, action) => { state.push(action.payload); },
    deleteExpense: (state, action) =>
      state.filter(item => item.id !== action.payload)
  }
});
export const { addExpense, deleteExpense } = expenseSlice.actions;
const store = configureStore({
  reducer: { expenses: expenseSlice.reducer }
});
export default store;
```

```
// Wrap app in main.jsx
import { Provider } from "react-redux";
root.render(<Provider store={store}><App /></Provider>);

// Inside a component
const expenses = useSelector(state => state.expenses);
const dispatch = useDispatch();
// dispatch(addExpense({ id: Date.now(), description: "Bus", amount: 30 }));
```

Redux Toolkit uses Immer inside slice reducers, so mutation-like syntax there is safe. Do not mutate Redux state directly elsewhere.

# 9. Error Boundaries, JWT and HTTP codes

## Error Boundary pattern

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }
  static getDerivedStateFromError(error) {
    return { hasError: true };
  }
  componentDidCatch(error, info) {
    console.error("Caught:", error, info);
  }
  render() {
    if (this.state.hasError)
      return <h2>Something went wrong. Try again.</h2>;
    return this.props.children;
  }
}
// Usage: <ErrorBoundary><CourseDetails /></ErrorBoundary>
```

An Error Boundary catches rendering/lifecycle errors in descendant components and shows fallback UI. It does not catch ordinary event-handler errors, asynchronous callback errors or errors in the boundary itself.

## JWT login flow

• User submits credentials to a backend login endpoint over HTTPS.

• Backend verifies credentials and returns a signed JWT on success.

• Client stores the token according to the application's security design.

• Protected API request sends Authorization: Bearer .

• Backend verifies signature, expiry and authorization before returning protected data.

• Logout removes the client-side token/session and redirects to login.

```
const response = await fetch("/api/dashboard", {
  headers: { Authorization: `Bearer ${token}` }
});

// JWT structure: header.payload.signature
```

**Security:** JWT payload is encoded, not encrypted by default. Never put passwords/secrets in it. Client-side route guards are not sufficient; backend must validate every protected request. Prefer secure HttpOnly, SameSite cookies where suitable.

| HTTP | Meaning |
|---|---|
| 200 | OK — request succeeded. |
| 201 | Created — resource created. |
| 400 | Bad Request — invalid request data. |
| 401 | Unauthorized — missing/invalid authentication. |
| 403 | Forbidden — authenticated but not allowed. |
| 404 | Not Found. |
| 500 | Internal Server Error. |

# 10. Viva Q&A; — JavaScript

**Q. What is JavaScript?**
A. A high-level language used to add behaviour to web pages; it also runs outside browsers, for example in Node.js.

**Q. Difference between let, const and var?**
A. let/const are block-scoped; var is function-scoped. const prevents reassignment; let allows it.

**Q. What is an array?**
A. An ordered collection of values accessed using numeric indexes.

**Q. What is an object?**
A. A collection of key-value properties representing structured data.

**Q. map() vs forEach()?**
A. map() returns a new transformed array; forEach() executes a callback and does not return a transformed array.

**Q. filter() vs find()?**
A. filter() returns all matching elements; find() returns the first matching element or undefined.

**Q. What does reduce() do?**
A. Combines array elements into one accumulated result, such as a sum or object.

**Q. Does sort() mutate the array?**
A. Yes. sort() sorts in place. Use [...arr].sort(...) to sort a copy.

**Q. What is a callback?**
A. A function passed to another function to be called later or at a defined point.

**Q. What is a Promise?**
A. An object representing the eventual success or failure of an asynchronous operation.

**Q. What are Promise states?**
A. Pending, fulfilled and rejected.

**Q. What is async/await?**
A. Promise-based syntax that makes asynchronous code easier to read; an async function returns a Promise.

**Q. Why check response.ok?**
A. HTTP 404/500 responses do not normally reject fetch automatically; response.ok indicates success status.

**Q. What is the DOM?**
A. The Document Object Model: a tree-like programming representation of an HTML document.

**Q. What is an event listener?**
A. A registered function that responds to an event such as click, input or submit.

**Q. Why preventDefault()?**
A. To stop the browser's default action, such as reloading after form submission.

**Q. What is localStorage?**
A. Browser string key-value storage that persists across sessions for the same origin.

**Q. Why JSON.stringify/parse?**
A. stringify converts objects to JSON text; parse converts JSON text back to JS values.

**Q. What is a closure?**
A. A function that retains access to variables from its lexical outer scope.

**Q. What is inheritance?**
A. A mechanism where a class derives properties and methods from another class.

**Q. What do getters and setters do?**
A. They provide property-like access for reading and controlled assignment.

**Q. What does super() do?**
A. Calls the parent class constructor in a derived class; call it before using this.

# 11. Viva Q&A; — React and advanced topics

**Q. What is React?**
A. A JavaScript library for building component-based user interfaces.

**Q. What is a component?**
A. A reusable, self-contained piece of UI that can receive props and manage state.

**Q. What is JSX?**
A. HTML-like syntax in JavaScript that is transformed into JavaScript.

**Q. Props vs state?**
A. Props are read-only inputs from a parent; state is data managed by a component and can change.

**Q. What is component composition?**
A. Building a complex UI by combining smaller reusable components.

**Q. Why do list items need keys?**
A. Keys help React identify items that changed, were added or removed.

**Q. What is useState?**
A. A Hook that adds state to a function component and returns value plus setter.

**Q. What is useEffect?**
A. A Hook for synchronizing a component with external systems such as timers, subscriptions or fetching.

**Q. What does useEffect(..., []) mean?**
A. Runs after initial mount; cleanup runs on unmount. Development Strict Mode may run setup/cleanup an extra time.

**Q. What is a controlled component?**
A. An input whose value is controlled by React state through value/checked and onChange.

**Q. Why preventDefault in form submit?**
A. Stops normal form submission/page reload so React can validate and handle the data.

**Q. What is componentDidMount()?**
A. A class lifecycle method called after the component is inserted into the DOM.

**Q. Why clear setInterval()?**
A. To stop background work when no longer needed and prevent leaks or duplicate timers.

**Q. What is React Router?**
A. A routing library mapping URL paths to React UI without full page reloads.

**Q. What is a dynamic route?**
A. A route with a variable segment such as /students/:id; useParams() reads it.

**Q. What is Redux?**
A. A predictable state-management library for shared application state.

**Q. What is a reducer?**
A. A function that calculates next state from current state and an action.

**Q. What is dispatch?**
A. The function used to send an action to the Redux store.

**Q. What is Redux Toolkit?**
A. Recommended Redux tools that reduce boilerplate, including createSlice and configureStore.

**Q. What is an Error Boundary?**
A. A class component that catches rendering errors in descendant components and shows fallback UI.

**Q. Can Error Boundaries catch event handler errors?**
A. No. Handle event-handler/asynchronous errors with try/catch or Promise error handling.

**Q. What is JWT?**
A. JSON Web Token, a compact signed token often used for authentication/authorization claims.

**Q. Is JWT payload encrypted?**
A. Not by default; it is usually encoded and signed. Do not place secrets in it.

**Q. What is Authorization: Bearer?**
A. A standard header format for sending an access token: Authorization: Bearer .

**Q. Why validate JWT on backend?**
A. Client-side checks can be bypassed; the server must verify signature, expiry and permissions.

# 12. Final revision checklist and likely practicals

## Be ready to write these without notes

• A class with constructor, getter, setter and subclass using extends/super.

• map(), filter(), reduce(), find() and numeric sort() on an array of objects.

• A DOM click handler that reads input, validates, creates an li and appends it.

• Save/load an array using localStorage and JSON.stringify/parse.

• A fetch function using async/await, response.ok and try/catch.

• A closure hiding an array/balance and returning add/remove/total methods.

• A React component with props, map(), keys and conditional rendering.

• A useState counter and useEffect responding to state changes.

• A controlled form with validation and error messages.

• Routes for Home, Students, /students/:id, About and wildcard 404.

• A Redux slice with add/delete, configureStore, Provider, useSelector and useDispatch.

• An Error Boundary fallback and JWT protected API request pattern.

## Likely short practical questions

| Question | Concept to demonstrate |
|---|---|
| Find students who passed | filter() with marks >= 40 |
| Calculate total salary | reduce() over employee objects |
| Find highest-scoring student | sort descending or reduce |
| Increase salary by 10% | map() and object spread |
| Create private shopping cart | Closure with add/remove/total |
| Add task to list | DOM event, createElement, appendChild |
| Persist tasks after refresh | localStorage + JSON |
| Fetch city weather | async, fetch, response.ok, try/catch |
| Visitor counter | useState, functional updates, useEffect |
| Validate admission form | Controlled inputs and validation |
| Student details page | Dynamic route and useParams |
| Add/delete expense | Redux slice, dispatch, selector |
| Handle course component crash | Error Boundary |

## Final 30-minute revision

• 10 min: array methods, OOP, closures, DOM and localStorage.

• 10 min: React components, props/state, Hooks and forms.

• 5 min: Router, Redux, Error Boundary and JWT definitions.

• 5 min: speak answers aloud; explain one code example from each assignment.

**Viva formula:** define the concept → say why it is used → give one short example. If unsure, explain what you know instead of guessing.

This guide follows the topics and requirements in your four uploaded AWT assignments. Code blocks are compact revision examples, not complete implementations of the larger assignment applications.